

Processamento de Linguagem Natural

Diana Isabel de Sá Janeiro e Martins (2023218119)
João Frederico Correia da Costa Marques Lopes (2022231718)

Departamento de Engenharia Informática, Universidade de Coimbra

Abstract

O seguinte estudo tem como objetivo explorar como processamento de linguagem natural serve para a realização da tarefa de classificação de texto em português do Brasil, utilizando o mesmo corpus e problema definidos para três estudos diferentes. É apresentada a experimentação e respetiva comparação de resultados de várias abordagens de forma a analisar qual a abordagem que melhor supera este problema, segundo métricas estipuladas. Primeiramente, é criado um conjunto de regras gramaticais e semânticas; em segundo foi classificado o corpus com variados algoritmos de Machine Learning combinados com vetorizadores; e por último é aplicado prompt engineering a um Large Language Model pré-treinado.

1 Introdução

O Processamento de Linguagem Natural (PLN) representa uma das áreas mais dinâmicas e relevantes da Inteligência Artificial contemporânea, permitindo que máquinas analisem, compreendam e gerem linguagem humana de forma cada vez mais completa. Graças ao crescimento exponencial da disponibilidade de dados textuais e ao avanço das técnicas de aprendizagem automática, têm surgido soluções capazes de realizar tarefas complexas, como tradução, sumarização e, em particular, classificação de texto.

A classificação de texto é uma tarefa fundamental em PLN, consistindo na atribuição automática de uma categoria a um determinado excerto de texto com base no seu conteúdo. Esta tarefa é amplamente utilizada em contextos como deteção de notícias falsas, análise de sentimento ou filtro de *spam*, etc.

Quando aplicada a categorias próximas, como “citação”, “facto” e “viés”, a classificação torna-se um desafio. A dificuldade reside não só na escassez de dados anotados, mas também na sobreposição de características entre as classes, o que exige abordagens capazes de capturar as diferenças entre essas categorias.

É neste contexto que se insere o presente estudo, que tem como objetivo comparar três paradigmas distintos de classificação textual (criação de regras, vetorizadores combinados com *machine learning* e *prompt engineering*) aplicados ao mesmo corpus em português do Brasil, o *FactNews*, anotado precisamente nas três classes mencionadas. O objetivo

desta comparação é compreender quais as estratégias mais adequadas para lidar com a subtilidade das classes em análise, num *dataset* marcado pelo desequilíbrio entre classes e pela complexidade inerente ao discurso jornalístico.

2 Materiais

Este estudo foi servido, na sua totalidade, do o *dataset* ‘FactNews’, um *dataset* proposto por Francielle Vargas composto por 6,191 linhas. O mesmo é constituído pelos seguintes atributos: *file*(categórico); *id_sente* (numérico); *id_article* (numérico); *domain* (categórico); *year* (numérico); *sentences* (categórico); *classe* (numérico). Tem frases extraídas de notícias em português do Brasil retiradas de três fontes diferentes de notícias fidedignas como São Paulo, O Globo, e Estadão, e as suas anotações foram disponibilizadas junto dos dados que permitem classificar as frases em ‘citação’, ‘facto’ e ‘viés’.

2.1 Meta 1

Para o primeiro estudo as bibliotecas usadas aplicadas para predição são as bibliotecas ‘re’, ‘NLTK’, ‘pandas’ e ‘spacy’, todas ferramentas bastante completas e cruciais na análise de dados textuais.

Para construção e projeção de uma matriz de confusão que permite analisar métricas de precisão foram utilizadas as bibliotecas ‘scikit learn’ e ‘matplotlib’.

2.2 Meta 2

Neste segundo estudo, de entre as bibliotecas usadas foi aplicada para predição dos modelos a biblioteca scikit-learn, que é bastante completa e crucial em tarefas de classificação. Serve também para construção e projeção de uma matriz de confusão que permite analisar métricas de precisão e ‘matplotlib’ foi utilizado para construção de gráficos.

Finalmente, foi também recorrido à biblioteca ‘optuna’, para otimização de parâmetros.

2.3 Meta 3

A principal ferramenta motor deste terceiro estudo foi o Meta Llama 3.1, um Large Language Model pré-treinado pela

Meta AI. Este modelo de 8B parâmetros utiliza uma arquitetura transformer otimizada com Grouped-Query Attention (GQA), suportando contexto de 128k tokens e treinado em mais de 15 T de tokens de dados públicos multi linguísticos. Entre as línguas suportadas oficialmente encontra-se o português, tornando-o adequado para classificação dos textos presentes no dataset escolhido. A versão instruction-tuned do Llama 3.1 8B foi selecionada por ter sido otimizada via Supervised Fine-Tuning (SFT) e Reinforcement Learning with Human Feedback (RLHF).

3 Metodologia

De forma a, em seguimento das experiências, comparar e concluir sobre os três estudos, é feito, de forma uniforme uma divisão entre treino e teste. As proporções são de 70% do dataset para compor o conjunto de treino, e o conjunto de teste, guardado e comum aos três estudos, engloba os restantes 30%. A preservação do mesmo conjunto de teste para os diferentes estudos serve para preservar uma validação real dos mesmos. Caso contrário, as métricas de validação seriam poluídas.

3.1 Meta 1

Nesta meta foi desenvolvido um sistema de classificação de texto baseado em regras gramaticais e semânticas, sem recurso a algoritmos de aprendizagem computacional. O objetivo principal passou por explorar conhecimento linguístico extraído do corpus e transformá-lo em regras para a distinção das classes *citação*, *facto* e *viés*.

Num primeiro momento, foi realizada uma análise exploratória do corpus recorrendo a diferentes ferramentas de PLN, nomeadamente NLTK, spaCy e expressões regulares. Esta análise incluiu a extração de palavras mais frequentes por classe, identificação de categorias morfológicas, funções gramaticais, padrões de pontuação e outros indícios linguísticos relevantes presentes em cada categoria.

Com base neste conhecimento, foi então definido um sistema de regras com diferentes pesos para classificação. Cada regra utiliza combinações de padrões lexicais, categorias morfológicas e marcas de discurso (por exemplo, verbos, pronomes pessoais, entidades próprias ou vocabulário opinativo) para atribuir uma classe ao texto.

3.2 Meta 2

Na segunda meta foi desenvolvido um sistema de classificação de texto com recurso a métodos de aprendizagem computacional supervisionada. O objetivo foi treinar modelos a partir de representações vetoriais do texto.

O processo iniciou-se com um pré-processamento estruturado do corpus, incluindo tokenização, remoção de *stopwords* em português, normalização e técnicas de matização, complementado com *oversampling* para mitigar o desbalanceamento entre classes. Deu-se a divisão do conjunto de treino

nas proporções de 80% / 20% para o novo treino e também validação, garantindo um conjunto específico para afinação de hiperparâmetros.

Foram consideradas várias estratégias de representação: vetores baseados em frequência (CountVectorizer e TF-IDF) e vetores densos (Word2Vec treinado no próprio corpus e GloVe pré-treinado). Cada uma destas representações foi combinada com diferentes modelos clássicos de classificação supervisionada, incluindo Naive Bayes, SVM, KNN, Decision Tree, Random Forest, Redes Neurais e Regressão Logística.

A seleção de hiperparâmetros para os vetorizadores e classificadores foi realizada com recurso à biblioteca Optuna, recorrendo a *search* automático guiado pelo F1-score na validação.

3.3 Meta 3

Para melhor explorar a problemática da performance de Large Language Models no terceiro estudo, foram selecionados quatro tipos de prompt a aplicar ao LLM escolhido. São eles: Zero-shot; Few-shot; Chain-of-thought; e Chain-of-thought-zero-shot.

A abordagem do prompt zero-shot consiste num prompt que instruiu o modelo a classificar um certo texto diretamente numa de várias classes que lhe são dadas a conhecer, sem quaisquer exemplos. Neste prompt, o modelo recorre apenas ao conhecimento que adquiriu do pré treino.

Na abordagem few-shot, a prompt passa a incluir um curto conjunto de exemplos representativos do corpus, cada um composto por texto e rotulado com a respectiva classe, seguido do novo texto a classificar. Este tipo de prompt procura explorar a capacidade dos LLMs aprenderem através de contexto.

Já a abordagem de chain-of-thought introduz, adicionalmente, uma instrução para que o modelo explicita os passos do seu raciocínio antes de dar a sua resposta final. Para tal, o prompt inclui exemplos rotulados com uma breve justificação em linguagem natural, incentivando o modelo a decompor a decisão em vários aspetos, o que facilita a análise de erros e a interpretabilidade.

Finalmente, a abordagem chain-of-thought-zero-shot remove os exemplos rotulados do último, mantendo apenas a instrução para que o modelo explique a sua decisão antes de indicar a classe. Este prompt permite estudar se a indução de fazer um raciocínio estruturado, mesmo sem exemplos rotulados, é suficiente para melhorar as classificações em comparação ao zero-shot normal, e também permite compreender padrões nas justificações produzidas pelo modelo.

4 Experiências

4.1 Meta 1

Primeiramente, procedeu-se ao pré-processamento do dataset, que consistiu na tokenização dos dados de treino

com remoção das stopwords presentes no corpus *portuguese* da biblioteca NLTK.

Embora tenha sido considerada a remoção de exemplos da classe 0 (significativamente maior), essa técnica não foi aplicada, pois as regras de classificação apresentadas são resistentes classificações tendenciosas por desequilíbrio de classes.

De seguida, realizou-se uma análise exploratória para estabelecer regras preditivas. Foram identificadas as dez palavras mais frequentes por classe:

Classe -1: gente(33), vamos(20), ... (19), povo(18), coisa(14), nada(13), quero(12), certeza(12), completou(10), mim(10)

Classe 0: presos(7), negou(5), defende(5), m(5), posse(5), irá(4), principais(4), lista(4), oficial(4), crimes(4)

Classe 1: maracanã(9), kaká(7), chinaglia(6), atacante(6), vagner(5), ronaldinho(5), marcação(5), desgaste(5), escolha(4), bolsonarista(4)

Posteriormente, identificaram-se os 10 tipos de token com maior discrepância de presença média entre as classes 0 e 1:

Token	Classe 0	Classe 1	Diferença
NPROP	0.5861	0.5381	0.0480
V	0.2251	0.2563	0.0311
NUM	0.0384	0.0204	0.0179
N—AP	0.0375	0.0210	0.0165
ADJ	0.1327	0.1445	0.0118
ADV	0.0632	0.0727	0.0096
V—+	0.0351	0.0446	0.0095
N	0.4810	0.4720	0.0090
VAUX	0.0265	0.0335	0.0071
(	0.0160	0.0104	0.0057

O conjunto final de regras foi ajustado por tentativa-erro, caso a adição de uma nova regra piorasse a classificação, não era mantida. Para decidir o peso de cada regra, utilizou-se o mesmo processo.

Assim, as regras finais utilizadas foram as seguintes:

- **Listas de palavras:** Criaram-se listas de palavras exclusivas de cada classe a partir da análise estatística do conjunto de treino. Se uma palavra aparecer exclusivamente numa classe específica, a classificação é imediata.
- **Léxicos:** Desenvolveram-se léxicos para detetar subjetividade e objetividade, associados à atribuição de pontos às classes 0 (facto) e 1 (viés).
- **Pontuação:** Definiu-se uma lista de sinais de pontuação para atribuir pontos às classes -1 (citação) e 1 (viés).
- **Funções gramaticais:** Calculou-se o número médio de ocorrências de cada categoria gramatical por frase em cada classe. Para atribuir pontos a uma classe,

comparam-se esses valores médios com as ocorrências da frase a classificar.

4.2 Meta 2

Neste estudo, todos os modelos de machine learning foram inicialmente avaliados individualmente com cada modelo de representação. Após identificar os melhores classificadores, estes foram aplicados aos dados pré-processados segundo a Meta 1, com o objetivo de comparar os dois métodos de pré-processamento. Partindo da análise dos gráficos abaixo é possível comparar o desempenho dos vários modelos segundo duas métricas escolhidas: Accuracy, sendo simples e fácil de interpretar, e F1-score por ser mais rígida, já que leva em conta o equilíbrio entre falsos positivos e falsos negativos. Sendo que os modelos de representação baseados em frequência obtiveram melhores resultados do que aqueles baseados em semântica, surgiu o interesse em combinar os dois vetorizadores de matrizes esparsas Count Vectorizer + TF-IDF de modo a observar se surgiriam melhores resultados.

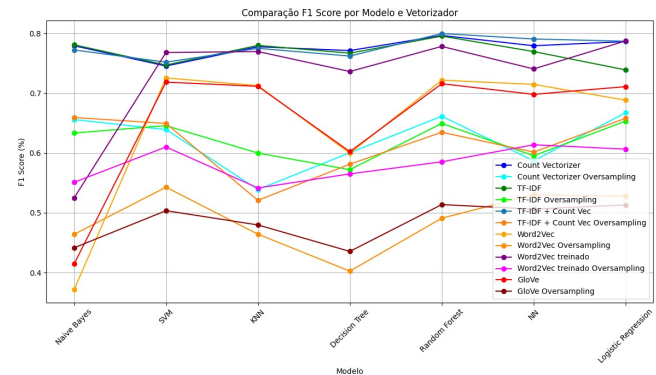


Figure 1: Comparação de f1-score entre os vetorizadores

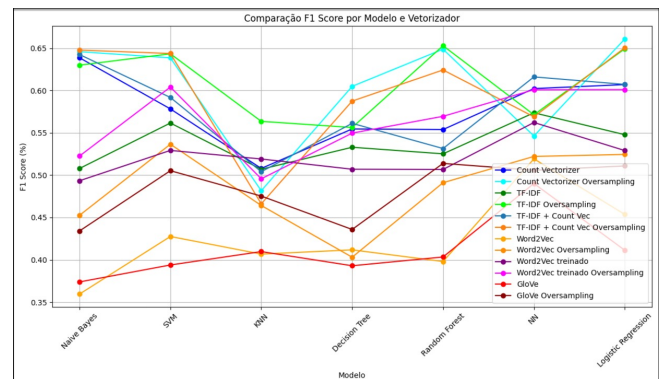


Figure 2: Comparação de accuracy entre os vetorizadores

4.3 Meta 3

4.3.1 Seleção de exemplos

Para selecionar exemplos representativos de cada classe, inicialmente, a abordagem passou por avaliar a entropia das pre-

visões de sete modelos de *machine learning* usando apenas o *CountVectorizer*, onde se observou que os valores não permitiam distinguir claramente exemplos que se destacassem. Testámos a mesma estratégia com conjuntos de treino balanceados e desbalanceados, incorporando mais vetorizadores e modelos. Apesar do aumento no número de previsões por exemplo, a entropia continuou a não ser uma métrica discriminativa por si só.

Explorou-se então uma abordagem com *K-means*, agrupando os exemplos com base nas suas representações vectoriais. Contudo, os *clusters* resultantes foram dominados pela classe maioritária. Além disso, como o *K-means* é um método não supervisionado, não considerava a classificação verdadeira, percebemos que era inadequado para seleccionar exemplos.

A solução final surgiu ao combinar a proximidade ao centróide de cada classe, calculado como o vetor médio no espaço do *CountVectorizer*, com a análise da entropia, tendo também em conta a taxa de acerto da classificação de cada exemplo. Para as classes 0 e -1, partimos dos 20 exemplos mais próximos do centróide e escolhemos os que apresentavam entropia mais baixa e maior taxa de acerto. Já para a classe 1, uma vez que os exemplos mais próximos do centróide não mostravam uma taxa de acerto satisfatória, flexibilizámos o critério de proximidade, indo além dos 20 iniciais e dando prioridade aos exemplos com melhor desempenho de classificação e menor entropia.

Desta forma, os exemplos são representativos da sua categoria (próximos do centróide), consensuais entre os diferentes classificadores (baixa entropia) e claramente identificáveis (alta taxa de acerto).

4.3.2 Prompts

Zero-shot Tu és um classificador de texto. Classifica o seguinte texto em português brasileiro:

{label_map}

Texto: {text}

Responde **APENAS** com -1, 0 ou 1 (sem mais texto).

Few-shot Tu és um classificador de texto. Classifica o seguinte texto em português brasileiro:

{label_map}

Tens aqui exemplos para auxiliar a classificação:

{examples}

Texto: {text}

Responde **APENAS** com -1, 0 ou 1 (sem mais texto).

Chain-of-thought Tu és um classificador de texto em português brasileiro.

Precisas de analisar o texto de forma lógica e passo a passo, pensando sobre o conteúdo, tom, intenção e significado antes de decidir a etiqueta correta.

Primeiro, raciocina internamente sobre o texto e explica brevemente (para ti) quais as pistas linguísticas e contextuais

que ajudam na classificação. Depois, com base nesse raciocínio, selecciona a etiqueta que melhor representa o texto.

Mapeamento de etiquetas: {label_map}

Exemplos para referência: {examples}

Texto: {text}

Quando terminares o raciocínio, responde **apenas** com um dos valores: -1, 0 ou 1 (Deve apresentar sem explicações ou texto adicional na resposta final.)

Zero-shot Chain-of-thought Tu és um classificador de texto em português brasileiro.

Precisas de analisar o texto de forma lógica e passo a passo, pensando sobre o conteúdo, tom, intenção e significado antes de decidir a etiqueta correta.

Primeiro, raciocina internamente sobre o texto e explica brevemente (para ti) quais as pistas linguísticas e contextuais que ajudam na classificação. Depois, com base nesse raciocínio, selecciona a etiqueta que melhor representa o texto.

Mapeamento de etiquetas: {label_map}

Texto: {text}

Quando terminares o raciocínio, responde **apenas** com um dos valores: -1, 0 ou 1 (Sem explicações ou texto adicional na resposta final.)

5 Resultados- acabar

5.1 Meta 1

Após as diferentes experiências feitas, finalmente foi obtido o seguinte resultado, com recurso à análise da matriz de confusão abaixo:

Classe	Precision	Recall	F1-score	Support
-1	0.4937	0.7597	0.5985	982
0	0.8499	0.3984	0.5425	2957
1	0.1385	0.5038	0.2172	395
Accuracy		0.4898		4334
Macro avg	0.4940	0.5539	0.3527	4334
Weighted avg	0.7044	0.4898	0.5255	4334

Table 1: Classification report

True label	Predicted label		
	-1	0	1
-1	746	104	132
0	673	1178	1106
1	92	104	199

Table 2: Matriz de confusão

Assim, é possível analisar que o modelo acerta razoavelmente bem a classe 0, mas tem dificuldades em distinguir entre as classes 0 e 1, principalmente porque grande

parte da classe 1 é classificada como 0 e vice-versa. A causa desta confusão dever-se-à às características linguísticas serem muito semelhantes entre estas classes e é difícil uma diferenciação puramente gramatical.

5.2 Meta 2

Optando escolher distinguir o melhor método segundo a métrica mais rígida (F1-score), foi possível observar que a melhor classificação foi feita partindo da combinação do Count Vectorizer com o modelo de Regressão Logística, o que demonstra que são uma combinação fidedigna para captar contexto e obter boa interpretação para o fim de classificação de texto. Desse modo, foram comparados os classification reports para os três tipos de conjuntos de dados: conjunto de dados da Meta 1; conjunto de dados do presente estudo; e conjunto de dados do presente estudo com oversampling. É possível observá-los em baixo: Fig. 3 Classification report, meta 1 Fig.4 Classification report Fig.5 Classification report, oversampling Assim os respectivos resultados são: F1-Score: 60.76 F1-Score c/oversampling: 65.02 F1-Score c/meta1: 59.05 Analisando os F1-scores em cada classe, é possível perceber que a técnica de oversampling resolveu uma disparidade entre o f1-score da classe 0 e da classe 1, pois tornam-se valores mais aproximados e o modelo, desta forma, tornou-se menos tendencioso.

Optando por distinguir o melhor método segundo a métrica mais rígida (F1-score), foi possível observar que a melhor classificação foi obtida com a combinação **CountVectorizer + Regressão Logística**, demonstrando ser uma abordagem fidedigna para captar contexto e obter boa interpretação na classificação de texto.

Foram comparados os *classification reports* para três configurações de dados:

Classe	Precision	Recall	F1-score	Support
-1	0.81	0.66	0.73	417
0	0.81	0.92	0.86	1273
1	0.28	0.13	0.18	168
Accuracy		0.79		1858
Macro avg	0.64	0.57	0.59	1858
Weighted avg	0.77	0.79	0.77	1858

Table 3: Classification report - Meta 1

Classe	Precision	Recall	F1-score	Support
-1	0.81	0.69	0.74	417
0	0.82	0.91	0.87	1273
1	0.30	0.17	0.22	168
Accuracy		0.79		1858
Macro avg	0.64	0.59	0.61	1858
Weighted avg	0.77	0.79	0.78	1858

Table 4: Classification report - Meta 1

Classe	Precision	Recall	F1-score	Support
-1	0.83	0.72	0.77	1273
0	0.55	0.77	0.64	1273
1	0.65	0.46	0.54	1273
Accuracy		0.65		3819
Macro avg	0.67	0.65	0.65	3819
Weighted avg	0.67	0.65	0.65	3819

Table 5: Classification report - Presente estudo

Configuração	F1-Score
Meta 1	59.05%
Sem <i>oversampling</i>	60.76%
Com <i>oversampling</i>	65.02%

Table 6: Comparação de F1-Scores

Analisando os F1-scores por classe, a técnica de *oversampling* resolveu a disparidade entre as classes 0 e 1, aproximando os valores e tornando o modelo menos tendencioso.

5.3 Meta 3

Os resultados das classificações obtidos com as diferentes estratégias de *prompt engineering* são apresentados nas tabelas seguintes, onde se comparam as métricas de desempenho para cada abordagem. Cada *prompt* foi executada múltiplas vezes, sendo aqui apresentados os melhores resultados observados.

Classe	Precision	Recall	F1-score	Support
-1	0.469	0.496	0.483	417
0	0.774	0.860	0.815	1273
1	0.000	0.000	0.000	168
Accuracy		0.701		1858
Macro avg	0.415	0.452	0.433	1858
Weighted avg	0.636	0.701	0.667	1858

Table 7: Classification report do zero-shot

Accuracy: 0.7008
F1 (weighted): 0.6667

Classe	Precision	Recall	F1-score	Support
-1	0.667	0.436	0.528	417
0	0.754	0.937	0.836	1273
1	0.000	0.000	0.000	168
Accuracy		0.740		1858
Macro avg	0.474	0.458	0.454	1858
Weighted avg	0.666	0.740	0.691	1858

Table 8: Classification report do few-shot

Accuracy: 0.7400
F1 (weighted): 0.6910

Classe	Precision	Recall	F1-score	Support
-1	0.705	0.103	0.180	417
0	0.771	0.887	0.825	1273
1	0.166	0.327	0.220	168
Accuracy		0.660		1858
Macro avg	0.547	0.439	0.408	1858
Weighted avg	0.701	0.660	0.625	1858

Table 9: Classification report do Zero-shot Chain-of-thought

Accuracy: 0.6604
F1 (weighted): 0.6253

Classe	Precision	Recall	F1-score	Support
-1	0.784	0.209	0.330	417
0	0.741	0.953	0.834	1273
1	0.189	0.125	0.151	168
Accuracy		0.711		1858
Macro avg	0.571	0.429	0.438	1858
Weighted avg	0.701	0.711	0.659	1858

Table 10: Classification report do Chain-of-thought com exemplos

Accuracy: 0.7110
F1 (weighted): 0.6590

Os resultados foram semelhantes para todos os testes mas as *prompts* que incluem exemplos obtiveram f1-score ligeiramente acima das *prompts zero-shot*, como seria expectável já que estamos a fornecer contexto ao modelo. O melhor resultado de *F1-score* foi obtido com a abordagem de *few-shot*. Para esta abordagem, experimentou-se alterar ligeiramente a *prompt*, fornecendo mais contexto sobre as características distintivas de cada classe, o que resultou na melhoria apresentada na Tabela 11.

Prompt Tu és um classificador de texto. Classifica o seguinte texto em português brasileiro:

{label_map}

Pensa com critério e lógica sobre cada frase. Se a frase tem incerteza ou imparcialidade, provavelmente é viés; se tem aspas, provavelmente é citação; se contém informações concretas ou menciona fonte de informação, provavelmente é facto.

Repara nessas pistas sobre as frases e podes usar também os exemplos seguintes para referência:

{examples}

Texto para classificar: {text}

Responde **APENAS** com -1, 0 ou 1 (sem mais texto).

Classe	Precision	Recall	F1-score	Support
-1	0.668	0.728	0.697	415
0	0.836	0.879	0.857	1273
1	0.167	0.065	0.094	168
Accuracy		0.772		1856
Macro avg	0.557	0.557	0.549	1856
Weighted avg	0.738	0.772	0.752	1856

Table 11: Classification report do *few-shot* com contexto adicional sobre as classes.

Accuracy: 0.7716
F1 (weighted): 0.7522

Comparação de resultados Observa-se que a classe mais difícil de classificar é a classe 1 (viés), uma vez que é aquela que sistematicamente apresenta pior desempenho. Já a melhor classificada é a classe 0 (facto). Estas correspondem, respetivamente, à classe minoritária e à classe majoritária, contudo, é importante notar que tanto a classificação da meta 1 como a da meta 3 não dependiam de treino com os dados de treino, pelo que a meta que mais pode explicar esta discrepância entre as duas classes é a meta 2, na qual se aplicaram modelos de *machine learning*.

Ainda assim, o número reduzido de exemplos na meta 1 pode ter influenciado o facto de não se conseguirem identificar padrões suficientemente claros nessa classe, dificultando a seleção de exemplos verdadeiramente representativos. Os fracos resultados nesta classe podem também dever-se à sua proximidade semântica com a classe 0.

Por outro lado, entre as três metas, aquela que apresentou piores resultados foi a meta 1, enquanto a que obteve melhor desempenho foi a meta 3, que utilizou uma *prompt few-shot* mais contextualizada. No entanto, os resultados não diferiram substancialmente das abordagens da meta 3 quando comparados com a classificação obtida pelos modelos de *machine learning* da meta 2.

6 Conclusão

Este estudo comparou três abordagens distintas para a classificação de texto: regras linguísticas, métodos de *machine learning* e *prompt engineering* em LLMs, utilizando o corpus FactNews com as classes “citação”, “facto” e “viés”. O objetivo foi identificar qual estratégia melhor capta as nuances entre categorias, num contexto de desequilíbrio entre classes.

Os resultados demonstram que a abordagem com *prompt engineering* no modelo Llama 3.1 obteve o melhor desempenho global, destacando-se a variante *few-shot* com contexto adicional (F1 de 0,752). A abordagem de *machine learning* com *CountVectorizer* e Regressão Logística também apresentou resultados robustos, especialmente com *oversampling*, enquanto o sistema baseado apenas em regras linguísticas foi

o menos eficaz, revelando a dificuldade de perceber manualmente a complexidade do discurso natural. Notou-se ainda uma dificuldade transversal na classificação da classe “viés”, a mais afetada pelo desequilíbrio do corpus e pela sobreposição de características com a classe “facto”.

Em conclusão, os resultados mostram que o *prompt engineering*, especialmente na variante *few-shot*, é a abordagem mais eficaz para captar as nuances entre as classes do corpus FactNews. Embora os modelos de *machine learning* também tenham apresentado desempenho competitivo, sobretudo com *oversampling*, as regras linguísticas revelaram-se insuficientes para lidar com a complexidade do discurso jornalístico. Persistem dificuldades na classificação da classe “viés”, evidenciando a necessidade de datasets mais equilibrados e de métodos capazes de lidar melhor com categorias próximas.